

PocketSmart AI: Your Smart Budget & Recommendation Assistant

Project Description

Managing budgets across different life needs—like home decor, event planning, or jewelry shopping—can be overwhelming due to the wide variety of products, platforms, and price ranges. PocketSmart AI addresses this challenge through a GenAI-powered, cross-platform recommendation system that delivers personalized, budget-based suggestions for products and services. Using Gemini 1.5 Flash Pro, PocketSmart AI analyzes user preferences, budgets, and contextual needs to generate curated recommendations from popular platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, and more. The platform helps users plan interior designs, parties, and even select matching jewelry for special occasions—all within their defined budgets. PocketSmart AI transforms budgeting into an intelligent, user-friendly experience through its smart planners, adaptive forms, and recommendation engine that bridges multiple categories and e-commerce ecosystems.

Scenario 1: Home Interior Planning with Smart Budget Allocation

Users begin by selecting the Home Budget Planner. After entering their budget, they specify room types (e.g., Living Room, Kitchen, Bedroom) and quantities for lights, ceiling fans, dining tables, etc. Gemini 1.5 Flash Pro processes this data and recommends cost-effective options for each category across platforms like IKEA and Amazon. Recommendations are balanced across functionality, style, and price, ensuring users can decorate their spaces beautifully without overspending.

Scenario 2: AI-Based Party Budget Planning

In the Party Planner, users input their total budget, guest count, event type, and venue details. Based on these inputs, the AI allocates the budget proportionally across catering, decoration, and entertainment, sourcing options from vendors listed on Swiggy, Zomato, and even accommodation services like OYO. The system tailors suggestions based on event type (e.g., birthday, corporate, wedding), helping users organize a successful event without financial guesswork.

Scenario 3: Jewelry Recommendations for Occasions

The Jewelry Budget Planner allows users to enter a budget, select the occasion, and define style preferences. Users can optionally upload an outfit image for the AI to consider color

coordination and aesthetics. Gemini 1.5 Flash Pro analyzes the data and offers jewelry options from platforms like Amazon and Flipkart, tailored to occasion and style.This ensures that users receive elegant, matching jewelry suggestions that fit both their personal style and financial plan.

Architecture Overview

PocketSmart AI is an intelligent, modular platform designed to generate personalized, budget-aware recommendations across domains like home interiors, party planning, and jewelry selection. It integrates a lightweight Flask backend with Google's Gemini 1.5 Flash Pro, a powerful multimodal AI model that processes user inputs (text, numbers, and images) to deliver smart recommendations.

Core Technologies

- Flask: Backend service layer for request handling, routing, and session control.
- Gemini 1.5 Flash Pro: AI model used for understanding budget context, analyzing images, generating structured suggestions, and recommending platform-specific products.
- Third-party APIs (Amazon, Flipkart, IKEA, Zomato, etc.): Product and service data sourcing.
- Frontend (HTML/CSS + JS): Interactive user interface for input collection and AI output display.

Component-Wise Architecture

Component	Description
Frontend UI	Web interface built with HTML/CSS allows users to input budgets, preferences, and images. Displays tailored AI-driven recommendations for each category.
Flask Backend	Manages user sessions, routes (/generate-home, /generate-party, /generate-jewelry), and handles communication between UI and Gemini AI.
Gemini AI Layer	Processes inputs using Gemini 1.5 Flash Pro to:

[image]

Pre-requisites

1. IBM Cloud Account Setup: IBM Cloud Console (<https://cloud.ibm.com/>)
2. Watsonx Access and Configuration: Watsonx Overview
3. Python Basics: Python.org (<https://www.python.org/>)
4. FastAPI Framework: FastAPI Docs

Project Workflow

1. Gemini AI Setup & Initialization

- Activity 1.1: Set up Google Cloud access (or platform) with Gemini 1.5 Flash Pro API access.
- Activity 1.2: Configure API keys, usage limits, and test integration for multimodal support (text + image).
- Activity 1.3: Fine-tune prompts for budget interpretation and product recommendation generation.

2. Core Functionality Development

- Activity 2.1: Design Flask-based backend structure for routing, session control, and planner modules.
- Activity 2.2: Create endpoints for Home Planner, Party Planner, and Jewelry Planner.
- Activity 2.3: Integrate Gemini prompts and templates for each planner with support for text + optional image inputs.

3. Backend Implementation (app.py)

- Activity 3.1: Define Flask routes like /generate-home, /generate-party, /generate-jewelry.
- Activity 3.2: Organize backend using modular structure:
 - routes/: Endpoints
 - services/: Recommendation logic & AI calls
 - models/: Input/output schemas
- Activity 3.3: Setup CORS headers for frontend communication and implement basic session management.
- Activity 3.4: Include mock API calls or simulated scraping from Amazon, IKEA, Zomato, etc.

4. UI Development

- Activity 4.1: Build responsive HTML forms for:

- Home Interior Planner (room details, quantities)
- Party Planner (event type, budget, guests)
- Jewelry Planner (budget, occasion, outfit image upload)
- Activity 4.2: Add dynamic result display and formatting for recommendations using JavaScript.

5. Testing & Optimization

- Activity 5.1: Perform testing with real-world use cases across all three planners.
- Activity 5.2: Evaluate Gemini response quality, budget adherence, and platform accuracy.
- Activity 5.3: Optimize prompts and add validations to prevent input edge-case failures.
- Activity 5.4: Add fallback/default recommendations when AI returns insufficient results.

MILESTONE 1: Gemini AI Initialization

In this foundational stage, the objective is to establish and configure the core AI infrastructure required to power PocketSmart AI. The platform relies on Gemini 1.5 Flash Pro—Google's multimodal foundation model—to interpret user inputs, process budgets, and generate personalized recommendations. This milestone ensures that the development environment is equipped with authenticated access to Gemini's APIs and is capable of performing both text-based reasoning and image analysis (specifically for the Jewelry Planner). Proper completion of this setup is essential, as it enables all AI-driven functionalities like budget understanding, category-wise recommendations, and multimodal analysis to operate seamlessly.

Activity 1.1: Set up Google Cloud account and enable Gemini API

- Sign in or create a Google Cloud account via console.cloud.google.com.
- Accept all terms and conditions and click on Get API key
- You will get trending models here you can select the model here or you can set the model by go to the API Documentation.

[image]

Activity 1.2: Configure access and permissions

- You will get interface like this and now click on Create API key

[image]

- Once click on API key you will the api key and make sure to copy the API key

[image]

Activity 1.3: Validate Gemini API connectivity

- Make a sample prompt call using Python to confirm integration.

[image]

- Test both text-only and image + text prompts to ensure multimodal compatibility.

[image]

MILESTONE 2: Core Functionalities Development

This milestone focuses on building the core backend intelligence of PocketSmart AI using the Flask framework and Google's Gemini 1.5 Flash Pro foundation model. The primary goal is to develop a robust and modular system capable of accepting user budgets (and optional image inputs), intelligently interpreting the context, and returning domain-specific, platform-aware recommendations. The backend handles prompt generation, API communication with Gemini, modular planning logic (for Home, Party, and Jewelry), and optional fallback strategies when product matches are limited. All core logic is abstracted into a utility service file (`gemini_utils.py`) that handles prompt orchestration, budget formatting, domain segmentation, and image analysis for multimodal tasks.

File Explorer Structure

[image]

Activity 2.1: Fast API Initialization and import libraries

- Import all required libraries for the project

[image]

- Fast API Initialization

[image]

- Load `.env` environment

[image]

Activity 2.2: Main Functionality code routings

- `/generate-home`: Generates home interior recommendations (furniture, decor, lighting)

[image]

- get home_recommendations by using the function

[image]

- linking the websites like amazon,flipkar,ikea ect.... to get the products

[image] [image]

- /generate-party: Suggests catering, venue, and decoration options for events

[image]

- get home_recommendations by using the function

[image] [image]

- linking the websites like amazon,flipkar,ikea ect.... to get the products

[image]

- using the if condition to categorize the situation based products

[image] [image]

- /generate-jewelry: Accepts text + image inputs to return style-matched jewelry options.
- async def jewelry_planner(request: Request, current_user: UserInDB = Depends(get_current_active_user)):

```
"""Jewelry budget planner page"""
```

```
return templates.TemplateResponse("jewelry_planner.html", {"request": request, "u
```

- get home_recommendations by using the function

[image] [image]

- linking the websites like amazon,flipkar,ikea ect.... to get the products

[image]

Activity 2.3: Login and Register page routings

- Login page

/login: Authenticates user credentials and initiates a session upon successful login.

[image]

- Register Page

/register: Handles new user registration by accepting and securely storing user credentials.

[image]

- Logout Page

/logout: Terminates the current user session and redirects to the login page.

[image]

Activity 2.4: Ensure modular code structure

- /token: Issues a secure JWT token after successful authentication, enabling authorized access to protected API routes.

[image]

- /session-info: Retrieves metadata about the current user session, such as user ID and login status.
- /session-data: Returns detailed session-specific data used for personalization and recommendation tracking.

[image]

MILESTONE 3: Backend – FastAPI Integration (main.py)

This milestone focuses on implementing the FastAPI-powered backend for PocketSmart AI. It acts as the bridge between the frontend interface and the Gemini-based AI recommendation engine defined in `gemini_utils.py`. The backend is designed with modular architecture, providing clean and maintainable API endpoints for planners across Home, Party, and Jewelry domains. It also includes features like CORS setup for cross-origin requests, session handling for personalized user experience, and structured input/output models to ensure smooth AI interaction and reliable frontend rendering.

- Activity 3.1: Define FastAPI Routes
 - Home Planner (/generate-home): Accepts home-related preferences and budget to return tailored product suggestions.

[image]

- Party Planner (/generate-party): Processes party details and guest count to recommend venue, food, and decoration items.

[image]

- Jewelry Planner (/generate-jewelry): Uses text and optional image inputs to suggest jewelry based on outfit and occasion.

[image] [image]

- User Auth (/register, /login, /logout, /token): Manages user registration, authentication, and session control.
- Session Info (/session-info, /session-data): Retrieves user-specific session data for personalized AI interaction.
- Activity 3.2: Modular Architecture Setup

/recommendations-details: Returns detailed AI-generated product recommendations based on user budget, preferences, and selected category (Home, Party, or Jewelry).

[image]

- Activity 3.3: CORS & Static Routing

/history: Retrieves the user's past recommendation queries and results for review or re-use.

[image]

- Activity 3.4: Startup and main function

/startup: Initializes essential application services and loads configuration settings when the FastAPI server starts.

[image]

main: Entry point of the FastAPI application that starts the server using uvicorn when the script is run directly.

[image]

MILESTONE 4: UI Development

This milestone focuses on developing a lightweight, responsive, and intuitive frontend for PocketSmart AI using HTML, CSS, and Jinja2 templates. The interface enables users to interact seamlessly with the system's budget-based recommendation capabilities across Home Interior, Party Planning, and Jewelry categories. It is tightly integrated with FastAPI backend endpoints to facilitate real-time input and AI-powered output rendering. The goal is to maintain a clean, user-friendly design that supports multi-category inputs, displays structured AI recommendations, and includes authentication and session-based personalization.

Activity 4.1: Develop HTML Frontend Interface

- Design a home page with budget input fields and category selectors (Home, Party, Jewelry)
- Implement individual forms for each planner module connected to FastAPI POST routes
- Display AI responses in well-structured, card-like layouts (e.g., product suggestions, item details, prices)

Templates Directory Structure:

[image]

MILESTONE 5: Testing & Optimization

This milestone focuses on validating the reliability, accuracy, and contextual awareness of PocketSmart AI's recommendation engine. The system is tested using real-world user scenarios across different planning categories—Home Interiors, Party Planning, and Jewelry Suggestions—to ensure consistent performance. Optimization efforts include refining AI prompt structures, improving session handling, and ensuring a smooth UI/UX across devices.

Activity 5.1: Test with Real-World Budget Inputs Across Domains

Conduct end-to-end testing using varied budgets and user preferences for:

- Home decor setups from platforms like IKEA, Amazon
- Party packages including food (Swiggy/Zomato), venues (OYO), and decor
- Jewelry suggestions matched with outfit styles, occasions, and image inputs

Home Page

Home Page: The main landing page introducing PocketSmart AI's features with links to planner modules and user actions.

[image]

Testimonials Page

Testimonials Page: Showcases real user reviews and success stories to demonstrate the platform's effectiveness.

[image]

Footer Part

[image]

Register Page

Register Page: Allows new users to create an account to access personalized budget recommendations.

[image]

Login Page

Login Page: Authenticates existing users to access their dashboard and saved data.

[image]

User Dashboard

User Dashboard: Displays recent recommendations, saved queries, and personalized insights for the logged-in user.

[image] [image]

Home Interior Budget Planner Page: Lets users input home decor preferences and budget to get tailored suggestions from multiple platforms.

[image]

Home Interior Recommendations Page: Displays AI-generated product suggestions for home decor based on the user's budget and style preferences.

[image]

Party Budget Planner Page: Helps users plan events by suggesting venues, food, and decorations based on budget and guest count

[image]

Party Budget Recommendations Page: Shows curated party planning options, including food, venues, and decorations, matched to the provided budget.

[image]

Jewelry Budget Planner Page: Provides jewelry recommendations aligned with occasion and outfit style, using text and optional image inputs.

[image]

Jewelry Budget Recommendations Page: Presents personalized jewelry options tailored to the occasion, outfit, and budget, using Gemini-generated insights.

[image]

Recommendation History Page: Displays a log of the user's past recommendation queries and results, allowing easy review and reuse of previous plans.

[image]aq

Conclusion

PocketSmart AI redefines the way individuals plan budgets for everyday lifestyle needs by delivering smart, AI-driven recommendations across home interiors, party planning, and jewelry selection. By combining the power of FastAPI with Gemini 1.5 Flash Pro, the system intelligently processes user inputs—budgets, preferences, and even images—to generate accurate and context-aware suggestions from multiple trusted platforms like Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO and more. From backend architecture to AI integration and a responsive frontend, PocketSmart AI has been built with a focus on efficiency, scalability, and user-centric design. The FastAPI backend ensures modular, secure routing and robust session handling, while the Jinja2-powered frontend offers an intuitive and clean interface for users to interact with the system in real time.

Each module—from registration to recommendations—is designed to simplify decision-making and enhance the shopping or planning experience through automation and personalization. Whether it's finding the right wall art within budget or organizing a last-minute party, PocketSmart AI acts as a smart digital assistant, saving time and effort while optimizing outcomes. With its seamless fusion of generative AI and web technologies, PocketSmart AI stands as a practical example of how AI can enhance everyday decision-making, bringing intelligent recommendations to users' fingertips—budget-friendly, fast, and personalized.

(Comment on Login Page: <http://console.cloud.google.com>)